

The requested changes have been applied:

- Apply a BPE method for tokenization.
- Apply the attention mechanism before the GRU.
- Verify training and overfitting through a Validation Loss.

BPE for tokenization

The construction of vocabulary is no longer based on what was previously implemented `Idx_to_w` and `w_to_idx`, but on a pre-implemented SentencePiece library, although my propensity to build from scratch the library, this allowed to me to build a more standardized vocabulary, executable locally in about ten seconds, and focus on other parts of the project.

Below is a short tutorial of the use I made of it.

In the original function, which served as a construction for the vocabulary, an attribute was added to collect the SentencePiece model (and therefore the vocabulary generated by BPE).

The BPE training (token generation), is performed on the sentences in input and target through

```
spm.SentencePieceTrainer.train(  
    input=src_tmp_path,  
    model_prefix=src_prefix,  
    vocab_size=vocab_size,  
    model_type='bpe',  
    character_coverage=1.0,  
    bos_id=0,  
    eos_id=1,  
    unk_id=2,  
    pad_id=3)
```

`src_tmp_path` It is the *input* (same for eng and span) text file on which the tokenizer is trained. Inside there are all the sentences I use to "teach" the model how to break the text into tokens.

`src_prefix` is the basic name of the output files.

In the end they will be created:

```
src_prefix.model  
src_prefix.vocab
```

`vocab_size` → Number of tokens you want in the final vocabulary.

`model_type='bpe'` → Indicates the type of algorithm

`character_coverage=1.0`

Indicate how much of the characters in the dataset you want to cover, in this case all the characters will be used (100%)

We can pre-define token corresponding to index:

bos_id=0, beginning of sentence token
eos_id=1, end of sentence token
unk_id=2, used for words not present in the vocabulary
pad_id=3 used to make padding in sequences when they have different lengths

src_prefix.model It is therefore saved in the corresponding vocabulary

To get the new number of words (token) in the new vocabulary we use
in_lang.n_words = sp_src.get_piece_size()

To inspect the generated tokens we can call

```
sp = input_lang.sp_model
for i in range(sp.get_piece_size()):
    print(i, repr(sp.id_to_piece(i)))
```

Below is an example print

0 '<s>'	383 '_yourself'	1000 '_forgive'	1480 'h'
1 '</s>'	384 '_ab'	1001 '_goodbye'	1481 'l'
2 '<unk>'	385 'ssed'	1002 '_himself'	1482 'm'
3 '<pad>'	386 '_des'	1003 '_laughed'	1483 'd'
4 '._'	387 '_did'	1004 '_overcon'	1484 'y'
5 '_t'	388 '_mad'	1005 '_quickly'	1485 'u'
6 '_i'	389 '_more'	1006 '_serious'	1486 'w'
7 '_s'	390 '_sleep'	1007 '_useless'	1487 'g'
8 're'	391 '_nothing'	1008 '_welcome'	1488 'c'
9 'he'	392 'han'	1009 '_winning'	1489 'p'
10 '_w'	393 '_en'	1010 '_worried'	1490 'k'

So each index corresponds to a token, possibly even a word, built through BPE.

So now each sentence will be converted to an idx_tensor and then operate an embedding on which to compute.

This preprocessing via BPE is implemented in
get_train_val_data loaders

Now get_train_val_data loaders in addition to the train_loader already used previously for training, it returns val_loader set also, in order to calculate the Loss validation in the future.

Attention mechanism

I implemented the attention mechanism reported by the book, deleting the previous additives and Luong attention, as it is the state-of-art and performs better than both.

Now the dataflow is:

EMBEDDING

$$e_t = \text{Dropout}(E(x_t))$$

MULTIHEAD ATTENTION (no more just a single head)

$$Q = e_t W_Q$$

$$K = M_t W_K$$

$$V = M_t W_V$$

For each head $h = 1, \dots, H$ the respective queries, key and value matrices

d_h = dimension of a single attention head

L_t = Number of tokens available up to time step t

$$Q_h \in \mathbb{R}^{1 \times d_h}, \quad K_h \in \mathbb{R}^{L_t \times d_h}, \quad V_h \in \mathbb{R}^{L_t \times d_h}$$

With which we calculate the scores

$$\text{scores}_h = \frac{Q_h K_h^\top}{\sqrt{d_h}}$$

The weights

$$\alpha_h = \text{softmax}(\text{scores}_h)$$

The context vector

$$C_h = \alpha_h V_h$$

We chain the results for each head

$$C = \text{Concat}(C_1, \dots, C_H)$$

And we use an output matrix to obtain a single projection vector

$$a_t = C W_O$$

GRU UNIT (stacked)

The GRU remains unchanged, but it is possible to add more than one to the process (stacked GRU)

$$z_t^{(1)} = [e_t; a_t]$$

$$h_t^{(1)} = \text{GRU}^{(1)}(z_t^{(1)}, h_{t-1}^{(1)}) \quad \text{first GRU layer}$$

$$h_t^{(l)} = \text{GRU}^{(l)}(h_t^{(l-1)}, h_{t-1}^{(l)}) \quad \text{if } l > 1$$

In the end the last hidden state

$$o_t = h_t^{(L)}$$

It will be used to calculate the logits

$$\text{logits}_t = o_t W_{\text{out}}$$

Then memory grows over time

$$M_{t+1} = \text{Concat}(M_t, h_t^{(L)})$$

So the decoder attends not only to encoder outputs (if those are inside memory), but also to **its own previous hidden states**.

Attention average across heads $\alpha = \frac{1}{H} \sum_{h=1}^H \alpha_h$ used mainly for the final plot.

Below is the complete process inside the new decoder:

$$\begin{aligned} e_t &= E(x_t) \\ a_t &= \text{MHA}(e_t, M_t) \\ h_t^{(1)} &= \text{GRU}^{(1)}([e_t; a_t], h_{t-1}^{(1)}) \\ h_t^{(l)} &= \text{GRU}^{(l)}(h_t^{(l-1)}, h_{t-1}^{(l)}) \\ \text{logits}_t &= W_{\text{out}} h_t^{(L)} \\ M_{t+1} &= [M_t; h_t^{(L)}] \end{aligned}$$

The last logits together with o_t, α, M_{t+1} are returned.

All logits_{token} (for each generated token) would be merge together and used to compute our final prediction $\text{LogSoftmax}(\text{Logits})$, which will be use to compute our final Loss.

Training & Validation

In training for each `n_epoch`,

After having compute and back propagate our loss during a `single_epoch`

We will also calculate the Loss on the Validation set (10% of the original dataset) disabling the gradient computation.

Given a validation input sequence X and target sequence Y :

$$H^{\text{enc}}, S^{\text{enc}} = \text{Encoder}(X)$$

$$\hat{Y} = \text{Decoder}(H^{\text{enc}}, S^{\text{enc}}, Y)$$

H^{enc} They are the encoder outputs

S^{enc} The final state encoders

$\hat{Y}$ The decoders output logits

The validation loss is computed as:

$$\mathcal{L}_{\text{val}} = \text{NLLLoss}(\hat{Y}, Y)$$

RESULTS

ENG-SPAN 6000

We need to specify how much we want to trim the original dataset.

```
max_input_length = 5
```

How many merges using BPE we want to create on both vocabulary

```
how_many_marges = 6000
```

And define our dataloaders, the batch size, the input and target vocabulary, and how much from the dataset we want to use as validation set.

```
input_lang, output_lang, train_loader, val_loader, max_len = \
    get_train_val_dataloaders(batch_size=32,
                              lang1='eng', lang2='span',
                              max_length=max_input_length, valid_ratio=0.1,
                              num_merges=(how_many_marges, 'sp'))
```

Here the following output

```
118964 sentence pairs readed
reduced to 7256 sentence pairs
eng 2236
span 4083
eng vocab size after SentencePiece: 6000
span vocab size after SentencePiece: 6000
```

The hidden size (the embedding dimension) rise from original 32 to 256

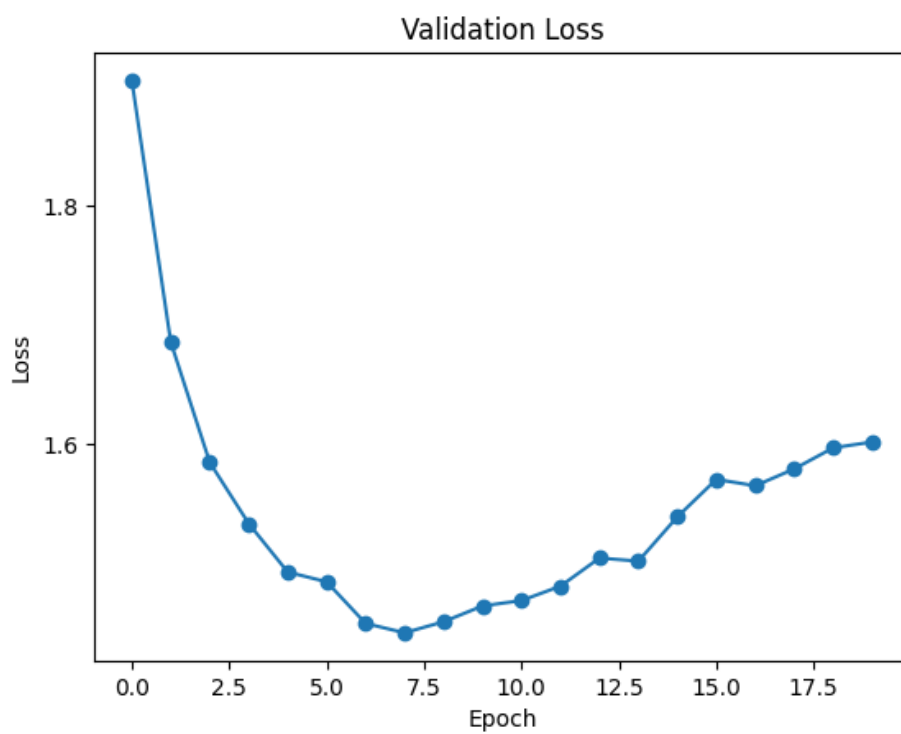
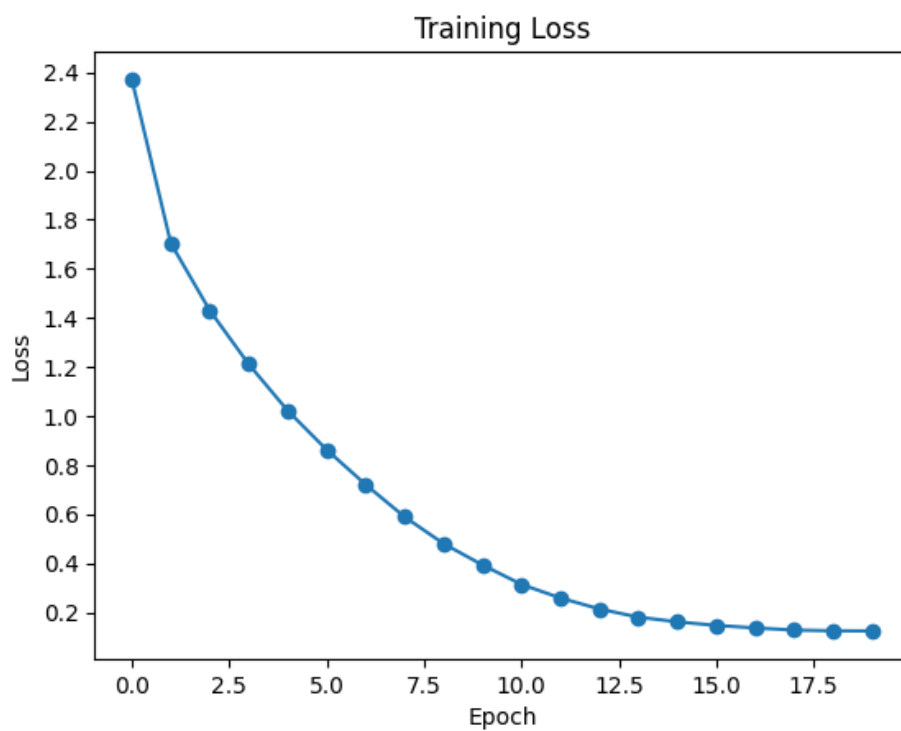
```
hidden_size = 256
```

Define the new encoder and decoder fixing the number of head attention (n_heads) and GRU layer (n_layers)

```
encoder = rnn_encoder(input_lang.n_words, hidden_size).to(device)
decoder = rnn_att_decoder(hidden_size, output_lang.n_words,
                          max_length=input_lang.max_len, n_heads=4,
                          n_layers=2).to(device)
```

Finally train with validation

```
train(train_loader, encoder, decoder,
      n_epochs=20, print_step=1, plot_step=1, valid_data_loader=val_loader)
```



With the setting defined above these are our results on training and validation Loss, after 20 epochs.

The trained parameters, are x10 more than the last time.

Encoder parameters:

embedding.weight 512000

gru.W_ih 196608

gru.W_hh 196608

gru.b_ih 768

gru.b_hh 768

Decoder parameters:

embedding.weight 512000

W_Q.weight 65536

W_K.weight 65536

W_V.weight 65536

W_O.weight 65536

gru_layers.0.W_ih 393216

gru_layers.0.W_hh 196608

gru_layers.0.b_ih 768

gru_layers.0.b_hh 768

gru_layers.1.W_ih 196608

gru_layers.1.W_hh 196608

gru_layers.1.b_ih 768

gru_layers.1.b_hh 768

out.weight 512000

out.bias 2000

Encoder parameters: 906752

Decoder parameters: 2274256

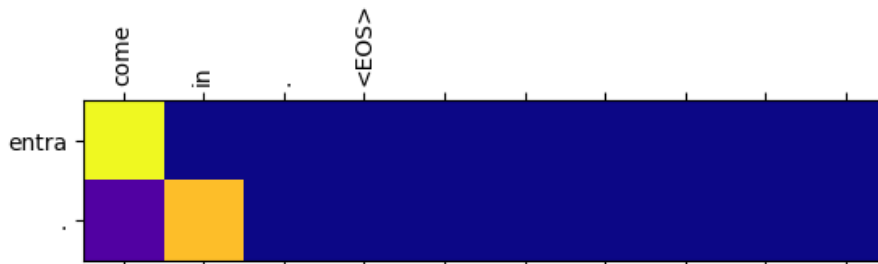
Total parameters: 3181008

Let's see some translation results (we really interested in).
Remember that we use just sentence of length ≤ 5 token (< . > dot included) so the model would be good in short translation.

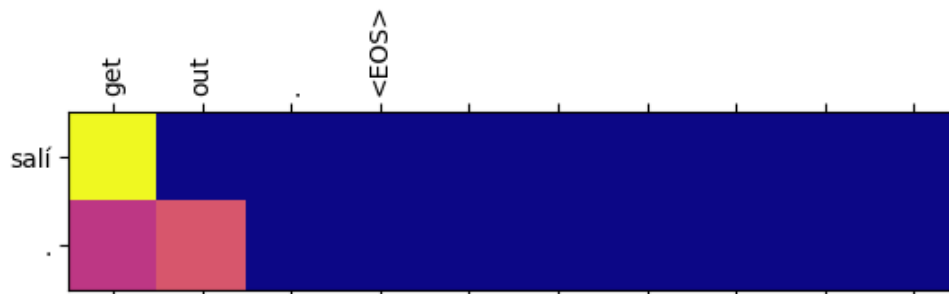
Let's see results on phrase **present** in the dataset.

```
evaluate_and_show_attention("come in .", encoder, decoder, input_lang, output_lang)
```

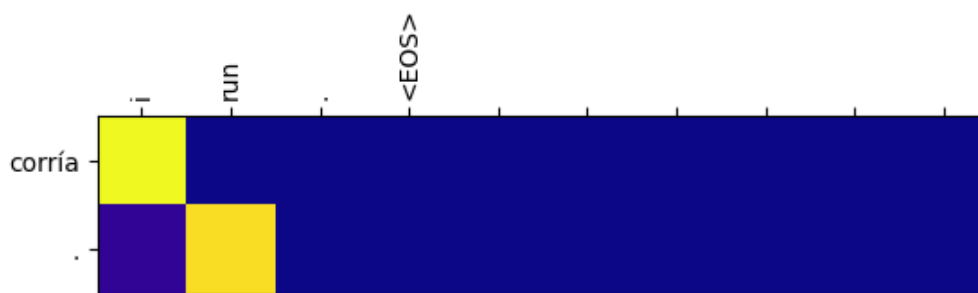
```
input = come in .  
output = entra .
```



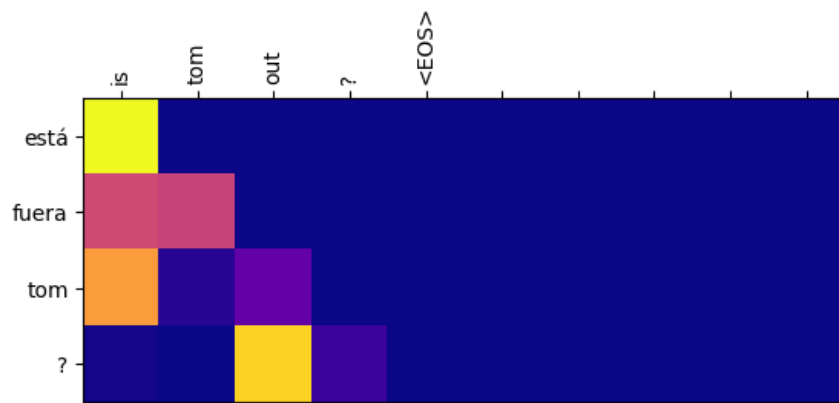
```
input = get out .  
output = salí .
```



```
input = i run .  
output = corría .
```



```
input = is tom out ?
output = está fuera tom ?
```

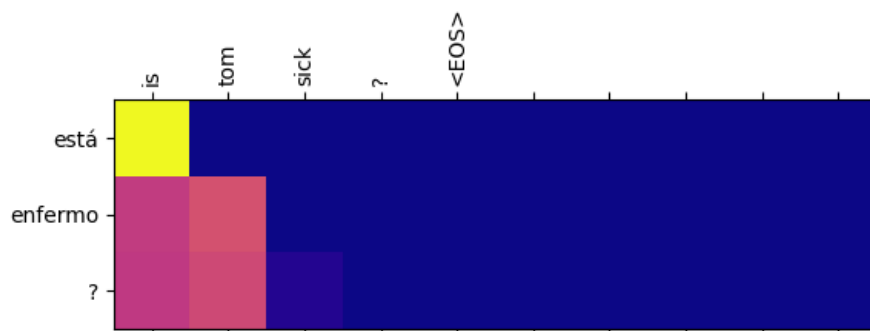


Good, so the baseline model, were we want to translate the phrase, already known from the training dataset, work!

Let's see if it still work for **similar** phrase, still taken from the dataset but not in the exact order or format as the original (something we cannot do in the previous code version).

```
original = tom is sick.
correct_original = Tom está enfermo.
```

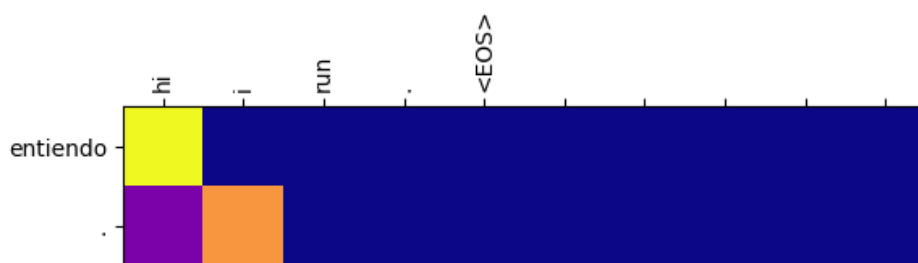
```
input = is tom sick ?
output = está enfermo ?
```



```
original = hi. + I run.
correct_original = Hola. + Corría.
```

So we expect "Hola corría." as output

```
input = hi i run .
output = entiendo .
```



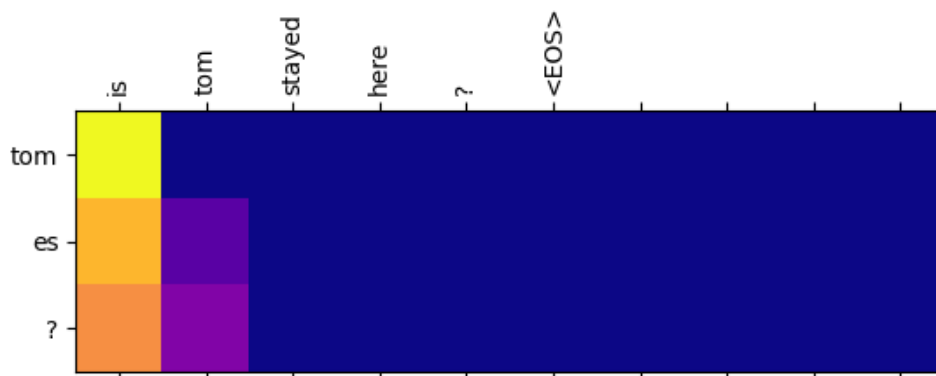
So the model got some *intuition*, in these cases attempting to get away from original set, it translate somehow correctly, but in other ones the translation sense completely lost, due to the fact that this model **heavily rely on the attention mechanism** and relationship **learned on training dataset**.

Let's try to mix more sentences

I stayed. Me quedé.
Is Tom in? ¿Está Tom?
I'm here. Estoy aquí.

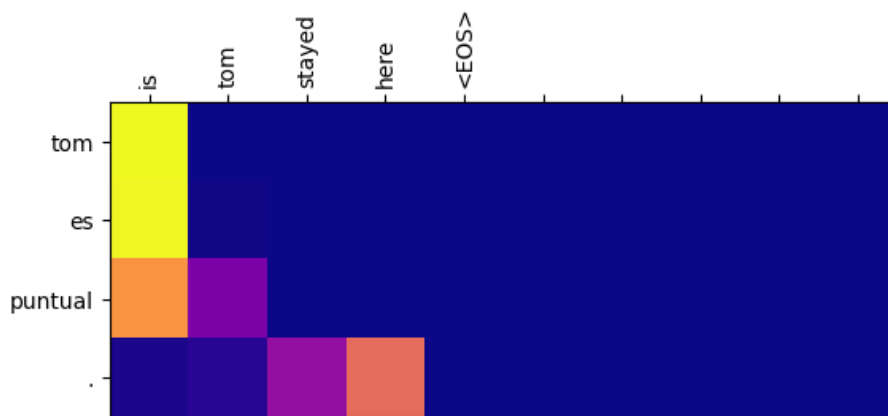
To create new one

input = is tom stayed here ?
output = tom es ?



And if we change it a little bit by deleting < ? >

input = is tom stayed here
output = tom es puntual .

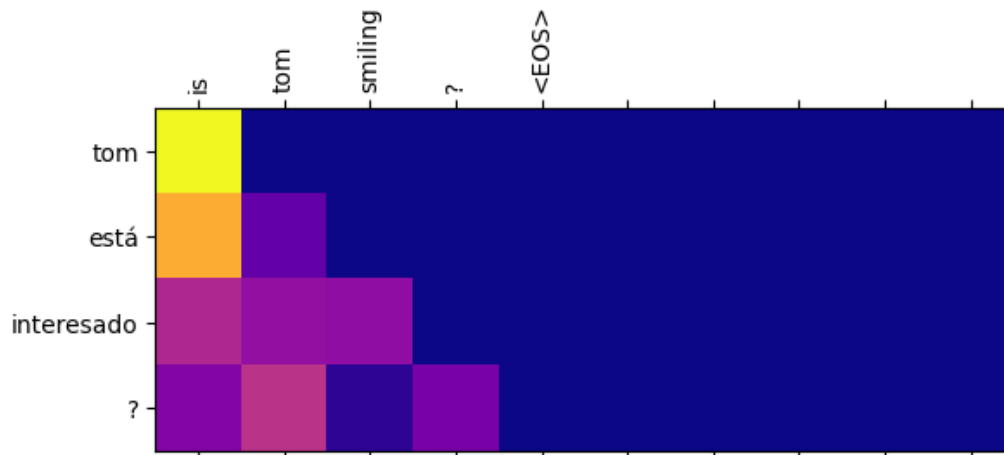


Mixing this time

Keep smiling. Sigue sonriendo.
Is Tom in? ¿Está Tom?

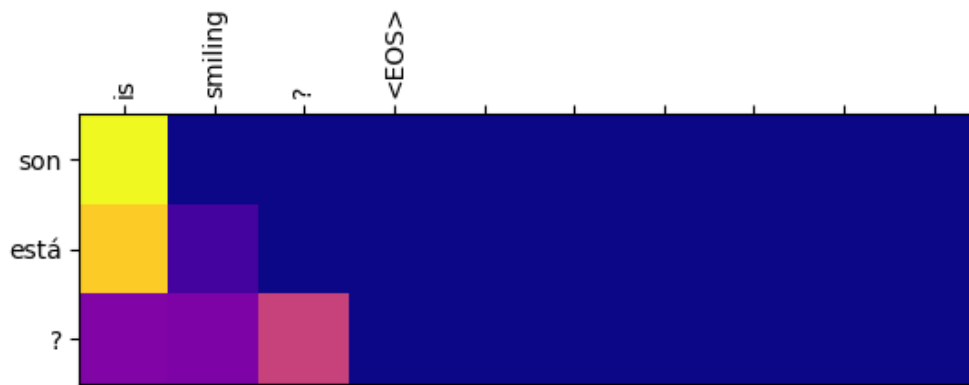
input = is tom smiling ?

output = tom está interesado ?



input = is smiling ?

output = son está ?



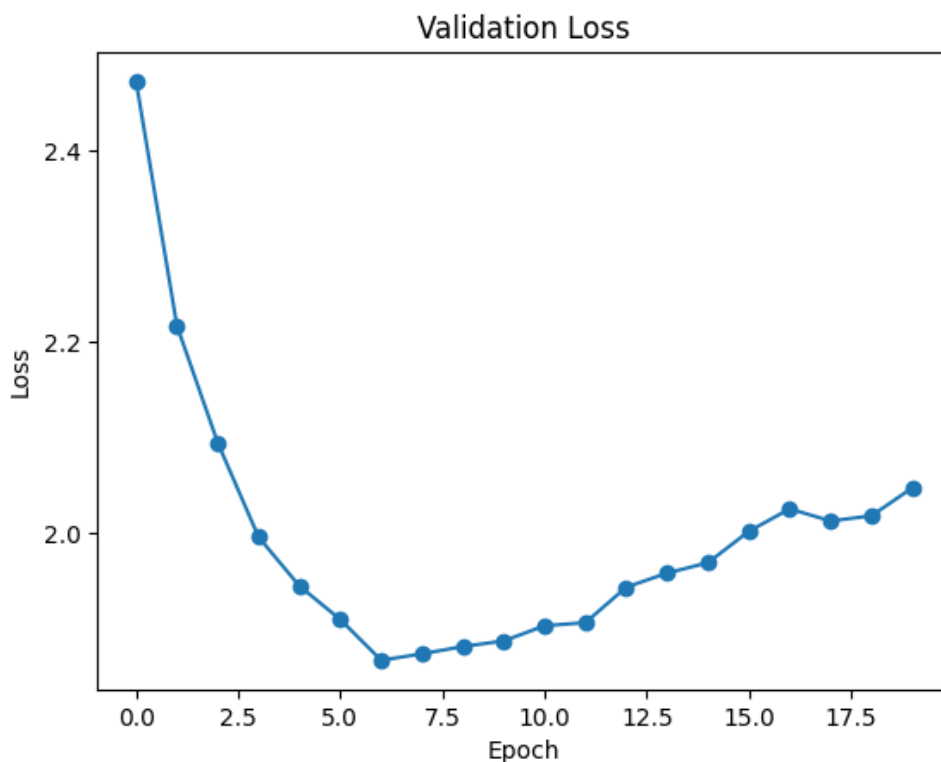
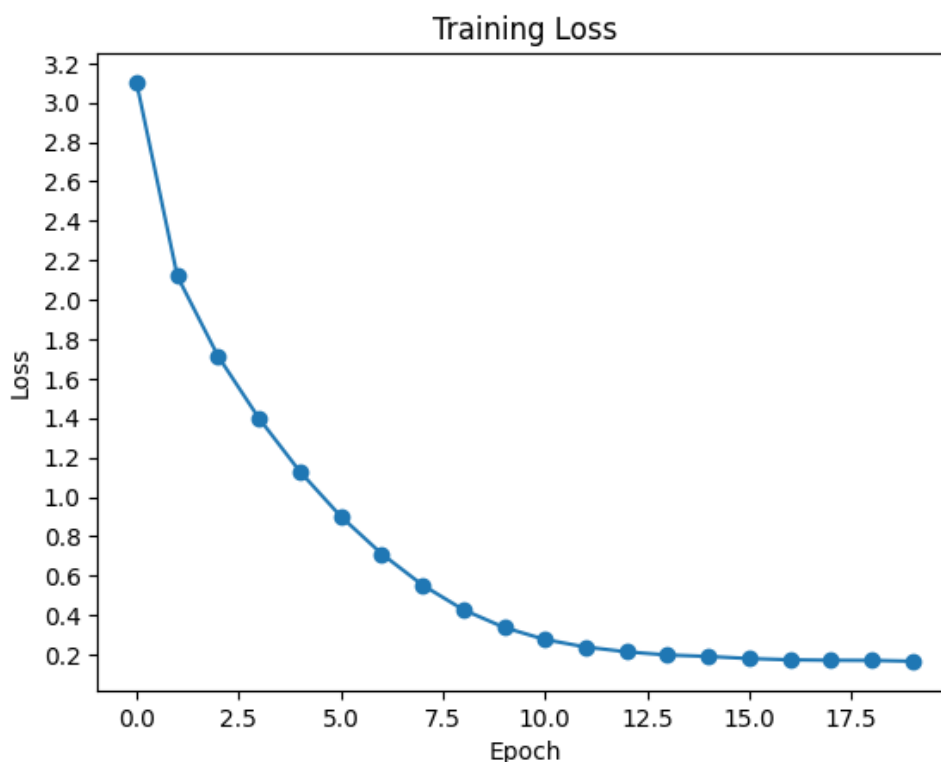
ENG-SPAN 8261

Now lets just change the `how_many_marges` parameter to 8261,

```
how_many_marges = 8261
```

which is the maximum number of merges possible in our English vocabulary dataset.
So the english vocabulary would be composed by
all the vocabulary word (original ones) + all possible subwords.

Repeat the training as before.



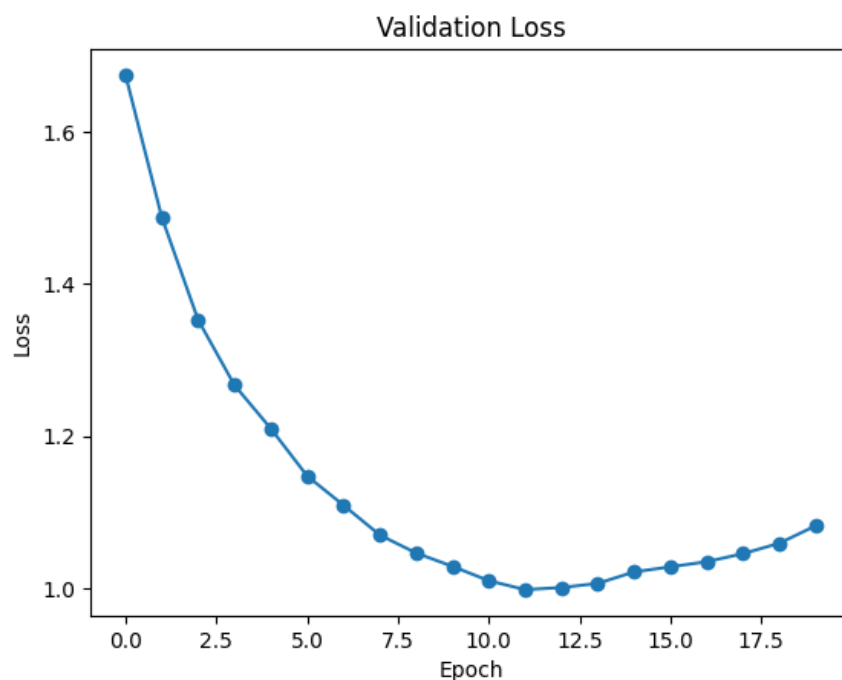
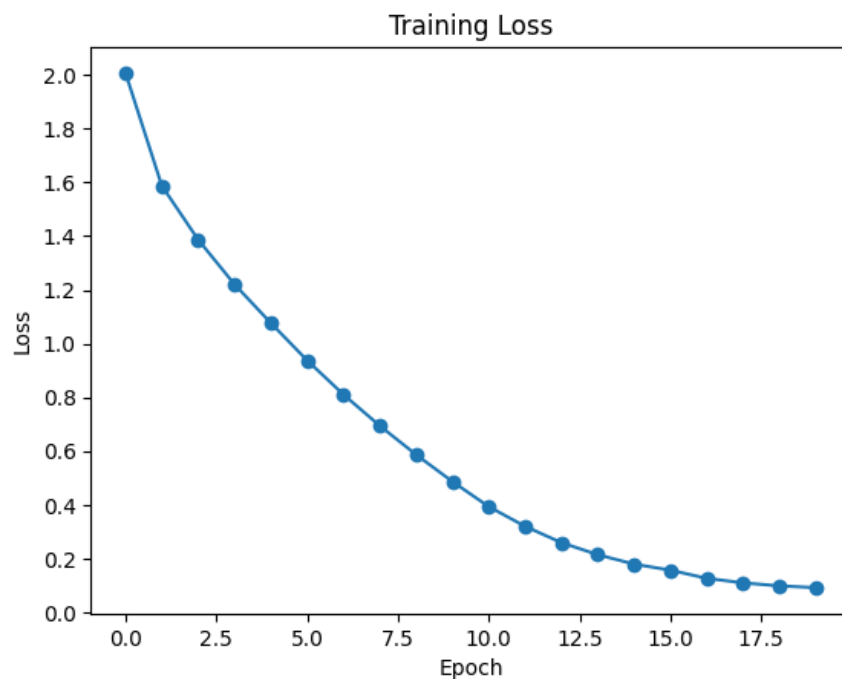
On the same number of epochs (20) we have not reach the same as good results as before on validation test, because the token embedding processed, and consequently all our attention weights, rely just on the original entire English words, and not on subwords, avoiding that much generalisation needed during evaluation.

The training performance are almost the same.

ENG-SPAN 1500

Now lets just change the `how_many_marges` parameter to 1500,

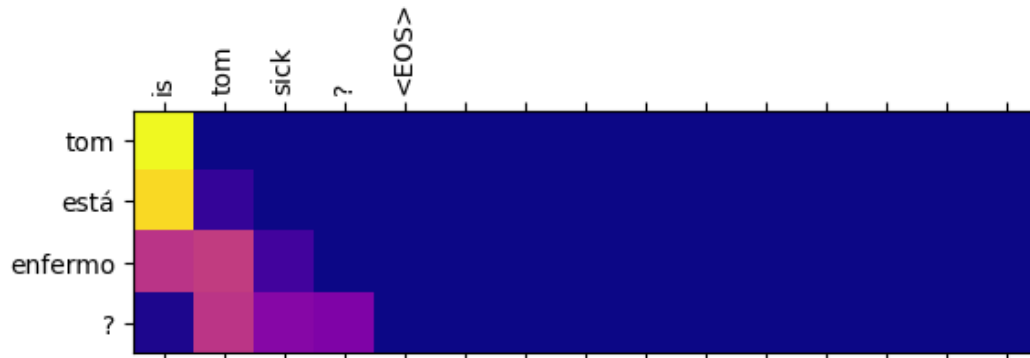
`how_many_marges` = 1500



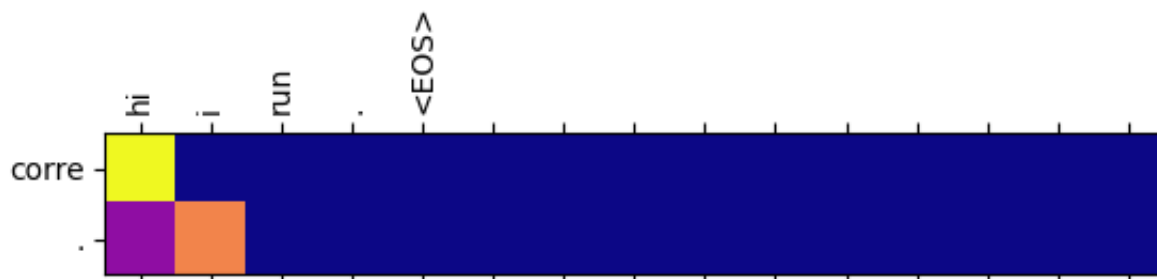
The dataset we have trained now, after BPE, is composed just on short frequent word + subword (e.g. prefix)
The model generalise better on validation set, getting the lowest loss until now.

In fact it perform better also on the previous **mixed** phrase results.

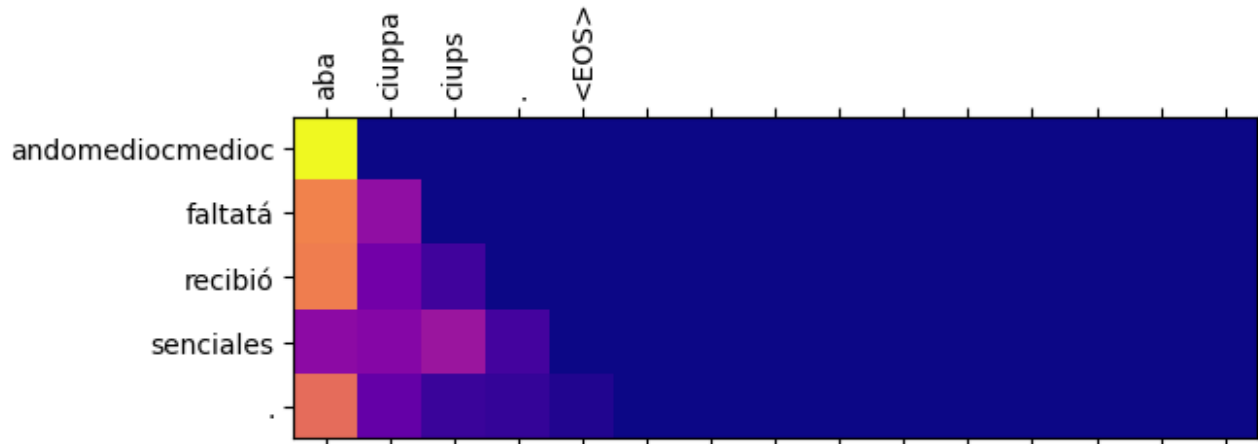
input = is tom sick ?
output = tom está enfermo ?



input = hi i run .
output = corre .



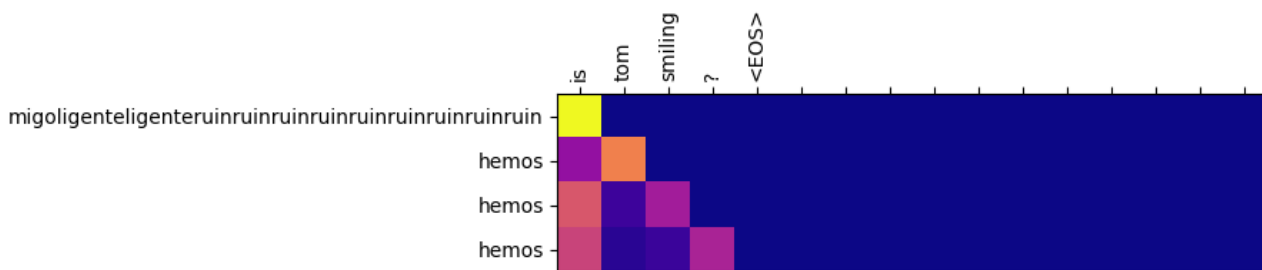
```
input = aba ciuppa ciups .
output = andomediocmedioc faltatá recibió senciales .
```



The model now generate new words “trying to guess the translation of random words”. (Surely *andomediocmedioc* is a concatenation of more subwords token).

The problem is the behaviour emerges also on our common word used in training.

```
input = is tom smiling ?  
output = migoligenteligenteruinruinruinruinruinruinruinruinruin ruin hemos hemos hemos
```



Here the problem of working with little subwords arise.

In order to reduce this noise sensitivity, and using less subword merges at the same time, we should also increment the number of `hidden_size`, GRU and attention head, stuck one over one other... creating a fully work transformer with many more weights... so building an LLM and no more a simple seq2seq as the starting idea...

And it will not keep just 30min for training.

Plus, we need to consider that the dataset itself for our translation task, is relative small, and every phrase appearing is shape to not having many repetition, being different every time.

All this circumstances not help to generalize and handle validation's phrase.

So we gonna stop ourself, and just create some other test on other language.

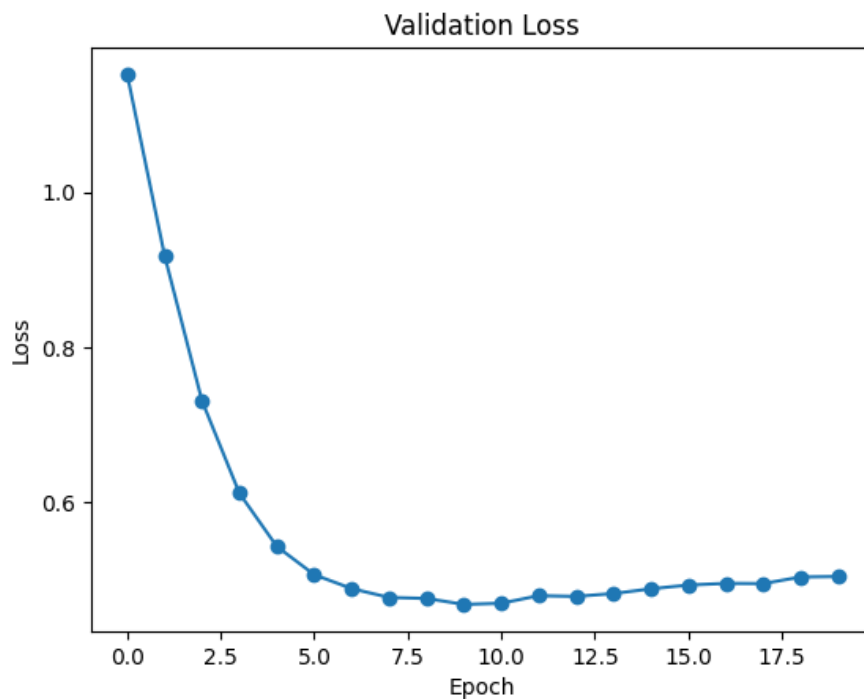
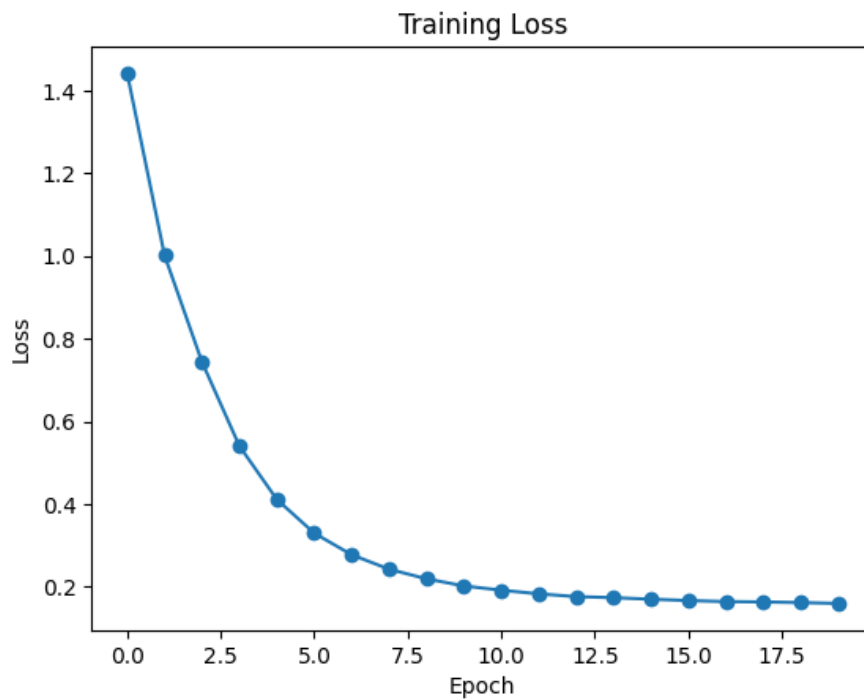
ENG-ITA 2500

In the eng-ita.txt we are working on larger dataset (4 times more).

400014 sentence pairs readed
reduced to 28287 sentence pairs
eng 3629
ita 7972

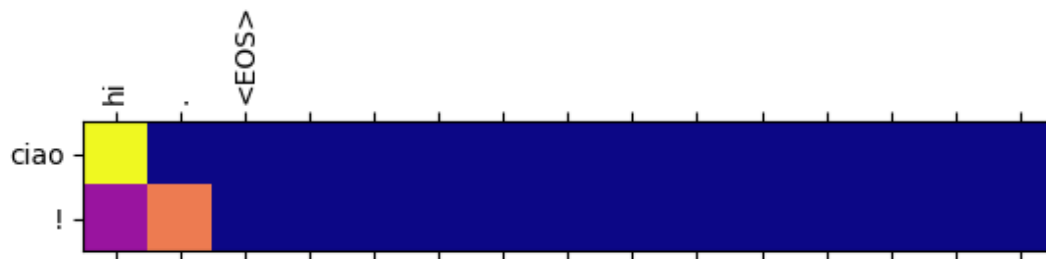
eng vocab size after SentencePiece: 2500
ita vocab size after SentencePiece: 2500

Training performance (20 epochs):



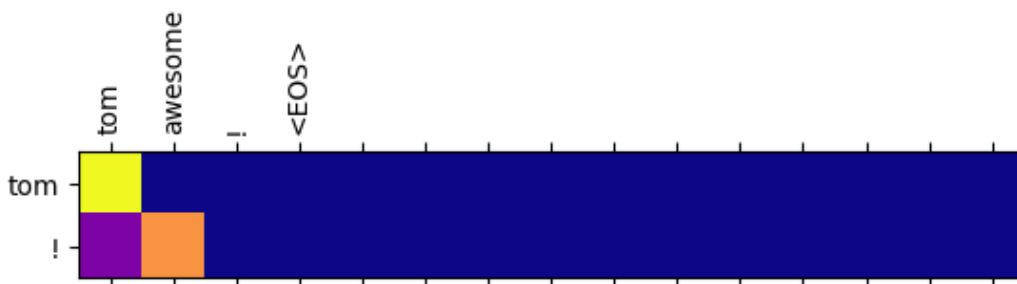
Evaluate on **original** phrase.

input = hi .
output = ciao !

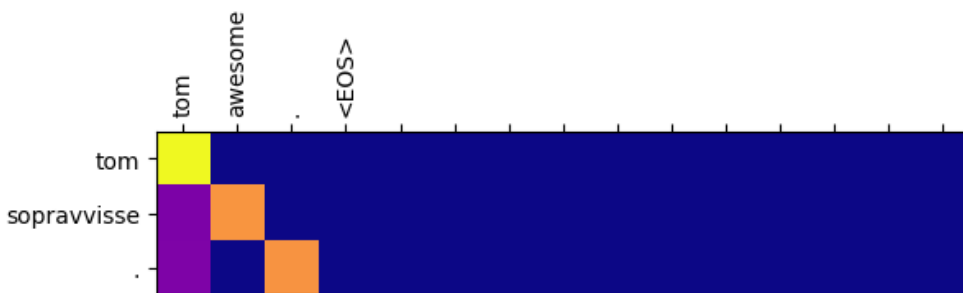


Evaluate on **similar** phrase.

input = tom awesome !
output = tom !

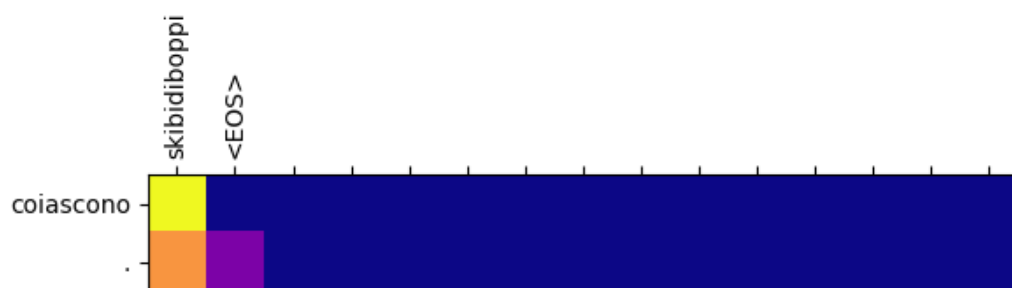


input = tom awesome .
output = tom sopravvisse .



Evaluate on **new** phrase.

input = skibidiboppi
output = coiascono .



The eng-ita dataset present the same difficulties of end-span dataset:

- able to translate from original phrase
- able somehow, on similar one
- Not at all on new one, creating new words

After 1h of training, we reach the better training and validation loss performance, due to the fact that we are training the model on 28287 (eng-ita) phrase respect to the previously 7256 (eng-span), four times more!